

Mục lục

Giới thiệu tổng quan	3
1 Lý thuyết về hàm của ma trận	7
1.1 Phương pháp trị riêng (Eigenvalue Methods)	7
1.1.1 Định nghĩa hàm ma trận qua dạng Jordan	7
1.1.2 Phương pháp vector riêng (Eigenvector Approach)	8
1.1.3 Phương pháp phân tích Schur	9
1.1.4 Độ nhạy của hàm ma trận	10
1.1.5 Mã nguồn Python minh họa	11
1.2 Phương pháp xấp xỉ (Approximation Methods)	12
1.2.1 Chặn sai số: Phân tích Jordan	12
1.2.2 Chặn sai số: Phân tích Schur	13
1.2.3 Xấp xỉ bằng chuỗi Taylor cắt cụt	13
1.2.4 Tính đa thức ma trận hiệu quả	14
1.2.5 Lũy thừa ma trận nhị phân (Binary Powering)	14
1.2.6 Tích phân hàm ma trận	14
1.2.7 Biểu diễn tích phân Cauchy	14
1.2.8 Mã nguồn Python minh họa	15
1.3 Hàm mũ ma trận (The Matrix Exponential)	16
1.3.1 Phương pháp xấp xỉ Padé kết hợp Scaling and Squaring	17
1.3.2 Tối ưu hóa tính đa thức Padé	17
1.3.3 Lý thuyết nhiễu loạn của hàm mũ ma trận	18
1.3.4 pseudospectra và hành vi quá độ	18
1.3.5 Phân tích ổn định của thuật toán Scaling and Squaring	19
1.3.6 Mã nguồn Python minh họa	19
1.4 Hàm dấu, căn bậc hai và logarit ma trận	21
1.4.1 Hàm dấu ma trận (Matrix Sign Function)	21
1.4.2 Căn bậc hai ma trận (Matrix Square Root)	22
1.4.3 Phân tích cực (Polar Decomposition)	23
1.4.4 Logarit ma trận (Matrix Logarithm)	24
1.4.5 Mã nguồn Python minh họa	25
2 Triển khai thực nghiệm	28
2.1 Môi trường và thiết lập thực nghiệm	28
2.1.1 Yêu cầu thư viện	28
2.1.2 Các ma trận thử nghiệm chuẩn	29
2.2 Thực nghiệm 1: Phương pháp trị riêng và Schur-Parlett	30
2.3 Thực nghiệm 2: Phương pháp xấp xỉ – Taylor và ảnh hưởng của hủy số	31
2.4 Thực nghiệm 3: Scaling and Squaring – phân tích hiệu năng theo kích thước	32
2.5 Thực nghiệm 4: Hàm dấu, căn bậc hai và logarit – hội tụ và độ chính xác	33
2.6 Tổng hợp và nhận xét kết quả	35

2.6.1	So sánh tổng quan các phương pháp	36
2.6.2	Khuyến nghị thực tế	36
2.6.3	Tổng kết	36

Giới thiệu tổng quan

Tính toán hàm $f(A)$ của một ma trận vuông $A \in \mathbb{R}^{n \times n}$ (hoặc $\mathbb{C}^{n \times n}$) là một bài toán xuất hiện phổ biến trong nhiều lĩnh vực ứng dụng: hệ thống điều khiển, phương trình vi phân ma trận, cơ học lượng tử, học máy và xử lý tín hiệu. Đây không chỉ là sự mở rộng hình thức của khái niệm hàm số thực lên không gian ma trận, mà còn là một lĩnh vực nghiên cứu sâu sắc với nhiều thách thức về lý thuyết lẫn tính toán số.

Bối cảnh lịch sử và tầm quan trọng

Lịch sử của hàm ma trận gắn liền với sự phát triển của đại số tuyến tính hiện đại. Vào cuối thế kỷ XIX, Arthur Cayley (1858) và William Rowan Hamilton đã đặt nền móng với định lý Cayley–Hamilton: mọi ma trận đều thỏa đa thức đặc trưng của chính nó, tức là $p(A) = 0$ với $p(\lambda) = \det(\lambda I - A)$. Từ đó, mọi hàm của A có thể được rút gọn thành đa thức bậc không quá $n - 1$, cung cấp một cơ sở lý thuyết quan trọng cho toàn bộ lĩnh vực.

Bước tiến lớn tiếp theo đến từ lý thuyết giải tích hàm (functional calculus), được phát triển mạnh trong nửa đầu thế kỷ XX. Frigyes Riesz và Béla Sz.-Nagy đã xây dựng lý thuyết phổ cho toán tử tuyến tính trên không gian Hilbert, từ đó lý thuyết hàm ma trận trở thành trường hợp hữu hạn chiều đặc biệt của một khung lý thuyết rộng lớn hơn nhiều. Đặc biệt, tích phân Cauchy cung cấp một cách định nghĩa thống nhất và tổng quát cho mọi loại hàm ma trận.

Về mặt ứng dụng thực tiễn, nhu cầu tính toán hàm ma trận tăng vọt từ những năm 1960–1970 khi máy tính điện tử trở nên phổ biến. Cleve Moler và Charles Van Loan (1978) tổng hợp 19 phương pháp tính hàm mũ ma trận, cho ra bài báo nổi tiếng “*Nineteen Dubious Ways to Compute the Exponential of a Matrix*” và cho thấy không có phương pháp nào hoàn toàn vượt trội. Công trình này đã thúc đẩy hàng thập kỷ nghiên cứu sau đó, đặc biệt bởi Nicholas Higham và các cộng sự.

Ý tưởng cơ bản

Nếu hàm số vô hướng $f(z)$ xác định trên phổ $\lambda(A)$ của A , thì $f(A)$ được xác định bằng cách thay “ A ” vào “ z ” trong công thức của $f(z)$. Ví dụ:

- Nếu $f(z) = \frac{1+z}{1-z}$ và $1 \notin \lambda(A)$, thì

$$f(A) = (I + A)(I - A)^{-1}.$$

- Hàm mũ: $e^A = \sum_{k=0}^{\infty} \frac{A^k}{k!}$.
- Hàm logarit: $\log(A)$ là ma trận X thỏa $e^X = A$.
- Căn bậc hai: $A^{1/2}$ là ma trận X thỏa $X^2 = A$.

Tuy nhiên, sự chuyển đổi từ hàm vô hướng sang hàm ma trận không phải lúc nào cũng đơn giản. Khi A là ma trận chéo hóa được, ta chỉ cần áp dụng f lên từng trị riêng. Nhưng khi A có trị riêng lặp hoặc không chéo hóa được, định nghĩa cần được mở rộng thông qua đạo hàm của f , dẫn đến lý thuyết về dạng chính tắc Jordan.

Bài toán trở nên thú vị và thách thức hơn khi f là hàm siêu việt (transcendental) như hàm mũ, logarit, lượng giác. Trong những trường hợp này, không tồn tại công thức đóng đơn giản; ta cần dùng đến các phương pháp số với phân tích sai số cẩn thận.

Các lĩnh vực ứng dụng

Sự quan trọng của hàm ma trận thể hiện rõ ở tính phổ biến của chúng trong nhiều ngành khoa học và kỹ thuật:

Phương trình vi phân Nghiệm của hệ phương trình vi phân tuyến tính $\dot{x}(t) = Ax(t)$, $x(0) = x_0$ có dạng $x(t) = e^{At}x_0$. Hàm mũ ma trận e^{At} do đó là trung tâm của lý thuyết hệ động lực tuyến tính.

Lý thuyết điều khiển Ma trận kiểm soát (controllability matrix), ma trận quan sát (observability matrix), và các bài toán tối ưu LQR đều liên quan đến hàm mũ ma trận. Phương trình Riccati ma trận, nền tảng của điều khiển tối ưu, được giải thông qua hàm dấu và phân tích cực ma trận.

Khoa học mạng (Network Science) Số mũ ma trận được dùng để đo tính kết nối và phân tích lan truyền thông tin trong mạng lưới. Centrality measures như communicability dựa trực tiếp trên e^A với A là ma trận kề của đồ thị.

Cơ học lượng tử Toán tử tiến hóa trong cơ học lượng tử có dạng $U(t) = e^{-iHt/\hbar}$, trong đó H là toán tử Hamiltonian. Việc tính toán hiệu quả ma trận e^{-iH} là bài toán trung tâm trong mô phỏng lượng tử.

Học máy và thống kê Trong học biểu diễn trên đa tạp Lie (Lie group learning), các phép tính trên không gian Riemannian đòi hỏi dùng hàm mũ và logarit ma trận. Hồi quy trên không gian đối xứng dương định (SPD matrices), quan trọng trong xử lý ảnh y tế và học sâu, dựa trên metric Riemannian với log và exp ma trận.

Tài chính định lượng Các mô hình tín dụng liên tục (continuous-time credit models) và mô hình xác suất chuyển tiếp (transition probability models) liên quan đến tính e^{Qt} với Q là ma trận cường độ (generator matrix) của xích Markov liên tục.

Phương trình vi phân đại số (DAEs) Hàm dấu ma trận được dùng để phân tách phổ của bó ma trận (matrix pencils), từ đó tách phần chỉ số hữu hạn và vô hạn trong DAEs.

Xử lý tín hiệu và hình ảnh Biến đổi ma trận theo hàm lũy thừa phân số A^α ($\alpha \in (0, 1)$) xuất hiện trong mô hình khuếch tán phân số (fractional diffusion), được ứng dụng trong nén ảnh và lọc tín hiệu thích nghi.

Thách thức tính toán

Mặc dù định nghĩa lý thuyết của hàm ma trận tương đối rõ ràng thông qua dạng Jordan hay tích phân Cauchy, việc tính toán số chính xác và hiệu quả đặt ra nhiều thách thức:

1. Khuếch đại sai số làm tròn. Do phép nhân ma trận tích lũy sai số, các phương pháp naive như chuỗi Taylor có thể cho kết quả sai hoàn toàn dù mỗi bước tính toán trông có vẻ đúng (hiện tượng “catastrophic cancellation”).

2. Tính không đơn trị. Hàm logarit và căn bậc hai ma trận không đơn trị: cùng một ma trận A có thể có nhiều logarit hoặc nhiều căn bậc hai. Việc chọn “nhánh chính” (principal branch) và đảm bảo thuật toán hội tụ về đúng nhánh đó đòi hỏi phân tích cẩn thận.

3. Trị riêng lặp hoặc gần nhau. Khi A có trị riêng lặp, các sai phân chia (divided differences) trong công thức Schur-Parlett có thể mất ổn định số. Cần dùng công thức thay thế dựa trên giới hạn (limit forms) của sai phân chia.

4. Ma trận không chính tắc (non-normal). Với ma trận không chính tắc, hàm mũ e^{At} có thể tăng trước khi suy giảm dù mọi trị riêng có phần thực âm — hiện tượng được gọi là *tăng trưởng quá độ* (transient growth). Phân tích phổ cổ điển không nắm bắt được hành vi này; cần dùng lý thuyết pseudospectra.

5. Chi phí tính toán. Với ma trận lớn (n hàng nghìn hoặc hàng triệu), chi phí $O(n^3)$ của các phương pháp dày (dense) trở nên không khả thi. Phương pháp Krylov subspace và phương pháp thưa (sparse methods) trở thành lựa chọn bắt buộc.

Tổng quan bốn hướng tiếp cận

Báo cáo này trình bày bốn hướng tiếp cận chính, mỗi hướng có ưu và nhược điểm riêng:

- Phương pháp trị riêng** (Phần 1.1): dựa trên phân tích Jordan và Schur. Phân tích Jordan cho định nghĩa nền tảng; phân tích Schur cho thuật toán Schur-Parlett ổn định số trong thực tế. Phương pháp này phù hợp khi cần tính chính xác cao với ma trận có kích thước vừa phải.
- Phương pháp xấp xỉ** (Phần 1.2): xấp xỉ f bởi hàm đơn giản hơn g (đa thức, hàm hữu tỉ). Bao gồm chuỗi Taylor cắt cụt, xấp xỉ Padé, và các kỹ thuật cầu phương. Hiệu quả và linh hoạt nhưng đòi hỏi phân tích sai số cẩn thận để tránh hủy số.
- Hàm mũ ma trận** (Phần 1.3): phương pháp scaling and squaring kết hợp xấp xỉ Padé là thuật toán “vàng” cho e^A , được dùng trong hầu hết các thư viện số hiện đại (MATLAB, SciPy). Phần này cũng phân tích lý thuyết nhiễu loạn và số điều kiện của hàm mũ.
- Hàm dấu, căn bậc hai và logarit** (Phần 1.4): các hàm này liên kết mật thiết với nhau qua lặp Newton và lặp Denman-Beavers. Phân tích cực ma trận cũng được trình bày như một ứng dụng quan trọng.

Ký hiệu và quy ước

Trong bài báo cáo ta dùng các ký hiệu sau:

- $\lambda(A)$: tập phổ (tập trị riêng) của A .

- $\kappa_2(X) = \|X\|_2 \|X^{-1}\|_2$: số điều kiện theo chuẩn 2.
- $\|\cdot\|_F$: chuẩn Frobenius.
- u : machine epsilon (độ chính xác máy tính), với IEEE 754 double precision $u \approx 2.22 \times 10^{-16}$.
- JCF: Jordan Canonical Form (dạng chính tắc Jordan).
- Q^H : liên hợp chuyển vị (Hermitian transpose) của Q .
- $\alpha(A) = \max\{\operatorname{Re}(\lambda) : \lambda \in \lambda(A)\}$: hoành độ phổ (spectral abscissa).
- $\sigma_i(A)$: trị kỳ dị (singular value) thứ i của A theo thứ tự giảm dần.
- $\rho(A) = \max\{|\lambda| : \lambda \in \lambda(A)\}$: bán kính phổ.

Chương 1

Lý thuyết về hàm của ma trận

1.1 Phương pháp trị riêng (Eigenvalue Methods)

1.1.1 Định nghĩa hàm ma trận qua dạng Jordan

Dạng chính tắc Jordan

Giả sử $A \in \mathbb{C}^{n \times n}$ có phân tích Jordan

$$A = X \cdot \text{diag}(J_1, \dots, J_q) \cdot X^{-1}, \quad (1.1)$$

trong đó mỗi khối Jordan có dạng

$$J_i = \begin{pmatrix} \lambda_i & 1 & \cdots & 0 \\ 0 & \lambda_i & \ddots & \vdots \\ \vdots & \ddots & \ddots & 1 \\ 0 & \cdots & 0 & \lambda_i \end{pmatrix} \in \mathbb{C}^{n_i \times n_i}, \quad i = 1, \dots, q.$$

Definition 1.1.1. Hàm ma trận (Jordan)

Hàm $f(A)$ được xác định bởi

$$f(A) = X \cdot \text{diag}(F_1, \dots, F_q) \cdot X^{-1},$$

trong đó

$$F_i = \begin{pmatrix} f(\lambda_i) & f^{(1)}(\lambda_i) & \cdots & \frac{f^{(n_i-1)}(\lambda_i)}{(n_i-1)!} \\ 0 & f(\lambda_i) & \ddots & \vdots \\ \vdots & \ddots & \ddots & f^{(1)}(\lambda_i) \\ 0 & \cdots & 0 & f(\lambda_i) \end{pmatrix}.$$

Định nghĩa này đòi hỏi tất cả các đạo hàm $f^{(k)}(\lambda_i)$ cần thiết phải tồn tại.

Biểu diễn chuỗi Taylor

Nếu f có thể biểu diễn bằng chuỗi Taylor trên phổ của A , thì $f(A)$ có cùng biểu diễn đó nhưng thay z bởi A .

Theorem 1.1.1. Biểu diễn chuỗi Taylor

Nếu f có biểu diễn Taylor

$$f(z) = \sum_{k=0}^{\infty} \frac{f^{(k)}(z_0)}{k!} (z - z_0)^k, \quad |z - z_0| < r,$$

và $|\lambda - z_0| < r$ với mọi $\lambda \in \lambda(A)$, thì

$$f(A) = \sum_{k=0}^{\infty} \frac{f^{(k)}(z_0)}{k!} (A - z_0 I)^k.$$

Bổ đề nền tảng cho định lý này xét trường hợp A là một khối Jordan đơn:

Lemma 1.1.1. Khối Jordan đơn

Giả sử $B \in \mathbb{C}^{m \times m}$ là khối Jordan, $B = \lambda I_m + E$ (E là phần bidiagonal trên nghiêm ngặt). Nếu $|\lambda - z_0| < r$, thì

$$f(B) = \sum_{k=0}^{\infty} \frac{f^{(k)}(z_0)}{k!} (B - z_0 I_m)^k.$$

Hơn nữa, nếu $0 \leq p \leq m - 1$, thì $[E^p]_{ij} = \delta_{i,j-p}$, và

$$f(B) = \sum_{p=0}^{m-1} \frac{f^{(p)}(\lambda)}{p!} E^p.$$

Các hàm ma trận quan trọng có biểu diễn chuỗi Taylor đơn giản bao gồm:

$$e^A = \sum_{k=0}^{\infty} \frac{A^k}{k!}, \quad \log(I - A) = - \sum_{k=1}^{\infty} \frac{A^k}{k} \quad (|\lambda| < 1),$$

$$\sin(A) = \sum_{k=0}^{\infty} \frac{(-1)^k A^{2k+1}}{(2k+1)!}, \quad \cos(A) = \sum_{k=0}^{\infty} \frac{(-1)^k A^{2k}}{(2k)!}.$$

Từ định nghĩa này ta có hai tính chất giao hoán quan trọng:

$$A \cdot f(A) = f(A) \cdot A, \quad f(X^{-1}AX) = X^{-1}f(A)X.$$

1.1.2 Phương pháp vector riêng (Eigenvector Approach)

Khi A là ma trận chéo hóa được: $A = X \operatorname{diag}(\lambda_1, \dots, \lambda_n) X^{-1}$, ta có hệ quả sau:

Corollary 1.1.1. Ma trận chéo hóa được

$$f(A) = X \cdot \operatorname{diag}(f(\lambda_1), \dots, f(\lambda_n)) \cdot X^{-1}.$$

Ví dụ minh họa

Xét

$$A = \begin{pmatrix} 1 + 10^{-5} & 1 \\ 0 & 1 - 10^{-5} \end{pmatrix}.$$

Ma trận vector riêng có $\kappa_2(X) \approx 10^5$. Với $u \approx 10^{-7}$, tính e^A qua đường chéo hóa cho kết quả sai ở chữ số thứ 2 của phân tử (1, 2), trong khi kết quả chính xác là $[e^A]_{12} \approx 2.718282$.

1.1.3 Phương pháp phân tích Schur**Ý tưởng chung**

Một số khó khăn của phương pháp Jordan có thể được khắc phục bằng cách dùng *phân tích Schur*: $A = QTQ^H$ (Q unitary, T tam giác trên). Khi đó:

$$f(A) = Q f(T) Q^H.$$

Vấn đề còn lại là tính $f(T)$ cho ma trận tam giác trên.

Công thức tường minh cho ma trận tam giác trên**Theorem 1.1.2. Hàm của ma trận tam giác trên – Schur**

Gọi $T = (t_{ij})$ là ma trận tam giác trên $n \times n$ với $\lambda_i = t_{ii}$, $F = f(T) = (f_{ij})$. Thì $f_{ij} = 0$ khi $i > j$, $f_{ii} = f(\lambda_i)$, và với $i < j$:

$$f_{ij} = \sum_{(s_0, \dots, s_k) \in S_{ij}} t_{s_0 s_1} t_{s_1 s_2} \cdots t_{s_{k-1} s_k} f[\lambda_{s_0}, \dots, \lambda_{s_k}],$$

trong đó S_{ij} là tập tất cả dãy tăng nghiêm ngặt từ i đến j , và $f[\lambda_{s_0}, \dots, \lambda_{s_k}]$ là *sai phân chia bậc k* (divided difference) của f .

Ví dụ cho 3×3 :

$$f \begin{pmatrix} \lambda_1 & t_{12} & t_{13} \\ 0 & \lambda_2 & t_{23} \\ 0 & 0 & \lambda_3 \end{pmatrix} = \begin{pmatrix} f(\lambda_1) & t_{12} \frac{f(\lambda_2) - f(\lambda_1)}{\lambda_2 - \lambda_1} & F_{13} \\ 0 & f(\lambda_2) & t_{23} \frac{f(\lambda_3) - f(\lambda_2)}{\lambda_3 - \lambda_2} \\ 0 & 0 & f(\lambda_3) \end{pmatrix},$$

trong đó F_{13} là tổ hợp tuyến tính của các sai phân chia bậc 1 và bậc 2.

Thuật toán Schur-Parlett

Parlett (1974) phát triển một phương pháp đệ quy hiệu quả dựa trên phương trình $FT = TF$. So sánh phần tử (i, j) của hai vế (với $t_{ii} \neq t_{jj}$):

$$f_{ij} = \frac{t_{ij}(f_{jj} - f_{ii})}{t_{jj} - t_{ii}} + \sum_{k=i+1}^{j-1} \frac{t_{ik}f_{kj} - f_{ik}t_{kj}}{t_{jj} - t_{ii}}.$$

Thuật toán 1.1.1 (Schur-Parlett) Tính $F = f(T)$ với T tam giác trên, trị riêng phân biệt.

Bước 1. Khởi tạo đường chéo: $f_{ii} = f(t_{ii}), i = 1 : n$.

Bước 2. Với $p = 1, 2, \dots, n - 1$ (siêu đường chéo thứ p):

$$\text{Với } i = 1 : n - p, \quad j = i + p : \quad f_{ij} = \frac{t_{ij}(f_{jj} - f_{ii}) + \sum_{k=i+1}^{j-1} (t_{ik}f_{kj} - f_{ik}t_{kj})}{t_{jj} - t_{ii}}.$$

Độ phức tạp: $\frac{2}{3}n^3$ phép tính dấu phẩy động (flops). Khi đó $f(A) = QFQ^H$.

Phương pháp Schur-Parlett theo khối

Khi A có trị riêng lặp hoặc gần nhau, sai phân chia trong Thuật toán 1.1.1 trở nên bất ổn định số. Giải pháp là dùng phiên bản *theo khối*:

1. Chọn Q sao cho T được phân hoạch thành các khối chéo $T_{11}, T_{22}, \dots, T_{pp}$ với $\lambda(T_{ii}) \cap \lambda(T_{jj}) = \emptyset$ và mỗi T_{ii} gắn với một cụm trị riêng.
2. Tính $F_{ii} = f(T_{ii})$ cho từng khối chéo (dùng phương pháp riêng cho cụm trị riêng).
3. Tính các khối ngoài đường chéo F_{ij} ($i < j$) qua hệ Sylvester:

$$F_{ij}T_{jj} - T_{ii}F_{ij} = T_{ij}F_{jj} - F_{ii}T_{ij} + \sum_{k=i+1}^{j-1} (T_{ik}F_{kj} - F_{ik}T_{kj}).$$

Hệ này được giải bằng thuật toán Bartels-Stewart.

1.1.4 Độ nhạy của hàm ma trận

Definition 1.1.2. Số điều kiện tương đối

Số điều kiện tương đối của f tại $A \in \mathbb{C}^{n \times n}$ là

$$\text{cond}_{\text{rel}}(f, A) = \lim_{\varepsilon \rightarrow 0} \sup_{\|E\| \leq \varepsilon \|A\|} \frac{\|f(A + E) - f(A)\|}{\|f(A)\|}.$$

Đây thực chất là chuẩn hóa của đạo hàm Fréchet của ánh xạ $A \mapsto f(A)$. Việc cài đặt cẩn thận thuật toán Schur-Parlett theo khối thường cho kết quả *ổn định tiến* (forward stable):

$$\frac{\|\hat{F} - f(A)\|}{\|f(A)\|} \approx u \cdot \text{cond}_{\text{rel}}(f, A).$$

Điều này *không thể* đảm bảo với phương pháp chéo hóa khi X kém điều kiện.

1.1.5 Mã nguồn Python minh họa

Đoạn mã dưới đây minh họa ba cách tính e^A : qua phân tích trị riêng (vector riêng), qua phân tích Schur (Schur-Parlett đơn giản hóa), và dùng thư viện SciPy. Kết quả so sánh cho thấy ảnh hưởng của số điều kiện $\kappa_2(X)$ lên độ chính xác.

```

1 import numpy as np
2 from scipy.linalg import expm, schur
3
4 # -----
5 # 1. Ma trận ví dụ: có số điều kiện cao
6 # -----
7 eps = 1e-5
8 A = np.array([[1 + eps, 1.0],
9               [0.0, 1 - eps]], dtype=float)
10
11 # -----
12 # 2. Phương pháp 1: Phân tích trị riêng (eigenvector)
13 # -----
14 eigenvalues, X = np.linalg.eig(A)
15 kappa = np.linalg.cond(X) # số điều kiện của X
16 f_eig = np.exp(eigenvalues) # áp dụng f = exp lên trị
    riêng
17 fA_eig = X @ np.diag(f_eig) @ np.linalg.inv(X)
18
19 # -----
20 # 3. Phương pháp 2: Phân tích Schur + công thức lặp
21 # -----
22 T, Q = schur(A, output='complex') # A = Q T Q^H
23 # Tính f(T) cho ma trận tam giác trên 2x2
24 fT = np.zeros_like(T)
25 lam = np.diag(T)
26 fT[0, 0] = np.exp(lam[0])
27 fT[1, 1] = np.exp(lam[1])
28 # Sai phân chia bậc 1 cho lặp (0,1)
29 if abs(lam[1] - lam[0]) > 1e-14:
30     fT[0, 1] = T[0, 1] * (fT[1, 1] - fT[0, 0]) / (lam[1] - lam[0])
31 else:
32     fT[0, 1] = T[0, 1] * np.exp(lam[0]) # giới hạn khi lambda trù
    ng nhau
33 fA_schur = Q @ fT @ Q.conj().T
34
35 # -----
36 # 4. Phương pháp 3: SciPy expm (Padé + thu gọn và bình phương)
37 # -----
38 fA_scipy = expm(A)
39
40 # -----
41 # 5. So sánh kết quả
42 # -----
43 print("="*55)
44 print(f" Số điều kiện của ma trận vector riêng: kappa = {kappa:.2e}

```

```

    })
45 print("="*55)
46 print("\nKt âqu qua phân tích ịtr riêng (eigenvector):")
47 print(fA_eig.real)
48 print("\nKt âqu qua Schur-Parlett đơ(n âgin hóa):")
49 print(fA_schur.real)
50 print("\nKt âqu SciPy expm (âchun xác):")
51 print(fA_scipy)
52
53 err_eig    = np.linalg.norm(fA_eig.real - fA_scipy, 'fro')
54 err_schur  = np.linalg.norm(fA_schur.real - fA_scipy, 'fro')
55 print(f"\nSai ỗs Frobenius (eigenvector vs scipy): {err_eig:.2e}")
56 print(f"Sai ỗs Frobenius (Schur      vs scipy): {err_schur:.2e}")

```

Listing 1.1: Phương pháp trị riêng: tính $f(A) = e^A$ và so sánh độ chính xác

1.2 Phương pháp xấp xỉ (Approximation Methods)

Ý tưởng cốt lõi: nếu $g(z) \approx f(z)$ trên $\lambda(A)$, thì $g(A) \approx f(A)$. Ví dụ:

$$e^A \approx I + A + \frac{A^2}{2!} + \cdots + \frac{A^q}{q!}.$$

1.2.1 Chặn sai số: Phân tích Jordan

Theorem 1.2.1. Sai số xấp xỉ – Jordan

Giả sử $A = X \operatorname{diag}(J_1, \dots, J_q) X^{-1}$ là JCF của A . Nếu f, g giải tích trên tập mở chứa $\lambda(A)$, thì

$$\|f(A) - g(A)\|_2 \leq \kappa_2(X) \cdot \max_{\substack{1 \leq i \leq q \\ 0 \leq r \leq n_i - 1}} \frac{|f^{(r)}(\lambda_i) - g^{(r)}(\lambda_i)|}{r!}.$$

Chặn này phụ thuộc trực tiếp vào số điều kiện $\kappa_2(X)$ của ma trận vector riêng.

1.2.2 Chặn sai số: Phân tích Schur

Theorem 1.2.2. Sai số xấp xỉ – Schur

Gọi $Q^H A Q = T = \operatorname{diag}(\lambda_i) + N$ là phân tích Schur của A , N là phần tam giác trên nghiêm ngặt. Nếu f, g giải tích trên tập lồi đóng Ω chứa $\lambda(A)$ trong nội bộ, thì

$$\|f(A) - g(A)\|_F \leq \sum_{r=0}^{n-1} \delta_r \frac{\|N\|^r}{r!}, \quad \delta_r = \sup_{z \in \Omega} |f^{(r)}(z) - g^{(r)}(z)|.$$

Remark 1.2.1. So sánh hai chặn

Chặn Jordan phụ thuộc $\kappa_2(X)$; chặn Schur phụ thuộc chuẩn của N (độ lệch chuẩn tắc). Với ma trận kém điều kiện eigensystem, chặn Jordan có thể quá bi quan. Ví dụ: nếu $\kappa_2(X) \approx 10^7$ thì chặn Jordan dự đoán sai số $O(1)$ trong khi chặn Schur cho $O(10^{-2})$ và sai số thực tế $\approx 10^{-5}$.

1.2.3 Xấp xỉ bằng chuỗi Taylor cắt cụt**Theorem 1.2.3. Sai số Taylor cắt cụt**

Nếu $f(z) = \sum_{k=0}^{\infty} \alpha_k z^k$ trên đĩa mở chứa $\lambda(A)$, thì

$$\left\| f(A) - \sum_{k=0}^q \alpha_k A^k \right\|_2 \leq \frac{n}{(q+1)!} \max_{0 \leq s \leq 1} \|A^{q+1} f^{(q+1)}(sA)\|_2.$$

Cảnh báo thực tế: Việc lấy nhiều số hạng hơn không nhất thiết tăng độ chính xác! Các tổng trung gian có thể có phần tử cỡ lớn, dẫn đến hủy số nghiêm trọng (catastrophic cancellation). Ví dụ với $A = \begin{pmatrix} -49 & 24 \\ -64 & 31 \end{pmatrix}$: lấy $q = 59$ cho lỗi lý thuyết $< 10^{-60}$, nhưng kết quả tính thực tế sai hoàn toàn vì các tổng trung gian có phần tử $\sim 10^7$ vượt ngưỡng máy tính $u \approx 10^{-7}$.

Kỹ thuật thu nhỏ (scaling)

Để khắc phục hạn chế của Taylor cắt cụt gần gốc tọa độ, người ta dùng các công thức nhân đôi góc. Ví dụ cho sin/cos:

$$\cos(2A) = 2\cos^2(A) - I, \quad \sin(2A) = 2\sin(A)\cos(A).$$

Từ đó xây dựng $\sin(A)$ và $\cos(A)$ từ các xấp xỉ Taylor cho $\sin(A/2^k)$ và $\cos(A/2^k)$ với k đủ lớn để $\|A\|_{\infty} \approx 2^k$.

1.2.4 Tính đa thức ma trận hiệu quả

Cần tính $p(A) = b_0 I + b_1 A + \cdots + b_q A^q$. Horner thông thường cần $q-1$ phép nhân ma trận. Tuy nhiên, thuật toán Paterson–Stockmeyer cho phép tính với $s+r-1$ phép nhân khi chọn $s = \lfloor \sqrt{q} \rfloor$, $r = \lfloor q/s \rfloor$:

$$p(A) = \sum_{k=0}^r B_k (A^s)^k,$$

giảm đáng kể số phép nhân ma trận.

Thuật toán 1.2.1 (Horner scheme cho đa thức ma trận)

Nhập: $A \in \mathbb{R}^{n \times n}$, hệ số $b_0, \dots, b_q$.

$$F \leftarrow b_q A + b_{q-1} I; \quad \text{Với } k = q-2, q-3, \dots, 0 : F \leftarrow AF + b_k I.$$

Độ phức tạp: $q-1$ phép nhân ma trận.

1.2.5 Lũy thừa ma trận nhị phân (Binary Powering)

Thuật toán 1.2.2 (Binary Powering) Tính $F = A^s$.

Viết $s = \sum_{k=0}^t \beta_k 2^k$ (khai triển nhị phân). Bắt đầu từ $Z = A$, bình phương liên tiếp và nhân có chọn lọc theo bit 1.

Độ phức tạp: tối đa $2 \lfloor \log_2 s \rfloor$ phép nhân ma trận.

1.2.6 Tích phân hàm ma trận

Để tính $F = \int_a^b f(At) dt$, áp dụng quy tắc cầu phương (ví dụ: Simpson):

$$\tilde{F} = \frac{h}{3} \sum_{k=0}^m w_k f(A(a + kh)),$$

với sai số bị chặn bởi

$$\|\tilde{F} - F\|_2 \leq \frac{nh^4(b-a)}{180} \max_{a \leq t \leq b} \|f^{(4)}(At)\|_2.$$

1.2.7 Biểu diễn tích phân Cauchy

Còn một cách định nghĩa $f(A)$ khác thông qua định lý tích phân Cauchy:

$$f(A) = \frac{1}{2\pi i} \oint_{\Gamma} f(z)(zI - A)^{-1} dz,$$

trong đó Γ là đường cong kín bao quanh $\lambda(A)$ và f giải tích trong và trên Γ . Phương pháp này, kết hợp với công cụ cầu phương bậc cao, cho phép tính toán một số loại hàm ma trận hiệu quả.

1.2.8 Mã nguồn Python minh họa

Mã nguồn dưới đây trình bày: (1) xấp xỉ Taylor cắt cụt và hiện tượng hủy số, (2) lược đồ Horner tính đa thức ma trận, (3) so sánh sai số theo bậc xấp xỉ q .

```
1 import numpy as np
2 from scipy.linalg import expm
3
4 # -----
5 # Hàm tính e^A bằng chuỗi Taylor cắt cụt bậc q
6 # -----
7 def expm_taylor(A, q):
```

```

8     """Áp dụng e^A bằng tổng Taylor bậc q."""
9     n = A.shape[0]
10    F = np.eye(n)          # ô s ạng k=0
11    Ak = np.eye(n)         # A^k
12    fact = 1.0
13    for k in range(1, q + 1):
14        Ak = Ak @ A
15        fact *= k
16        F += Ak / fact
17    return F
18
19 # Hàm tính đa thức ma trận p(A) bằng lược đồ Horner
20 def horner(A, coeffs):
21     """
22     Tính  $p(A) = \text{coeffs}[0]*I + \text{coeffs}[1]*A + \dots + \text{coeffs}[q]*A^q$ 
23     dùng lược đồ Horner.
24     coeffs[0] là hệ số tự do, coeffs[-1] là hệ số bậc cao nhất.
25     """
26     n = A.shape[0]
27     q = len(coeffs) - 1
28     F = coeffs[q] * np.eye(n)
29     for k in range(q - 1, -1, -1):
30         F = F @ A + coeffs[k] * np.eye(n)
31     return F
32
33 # -----
34 # Ví dụ 1: Ma trận Hessenberg
35 # A có trị riêng âm (n định, hàm mũ âm)
36 # -----
37 A_ok = np.array([[1.0, 2.0],
38                 [0.0, 3.0]])
39 ref = expm(A_ok)
40
41 print("Ví dụ 1 -- Ma trận Hessenberg A = [[1,2],[0,3]]")
42 print(f"  Két quả chuẩn xác (SciPy): \n{ref}\n")
43 for q in [5, 10, 20, 30]:
44     approx = expm_taylor(A_ok, q)
45     err = np.linalg.norm(approx - ref, 'fro')
46     print(f"  q={q:2d} ||Taylor - chính xác||_F = {err:.2e}")
47
48 # -----
49 # Ví dụ 2: Ma trận "ổng ngáo" -- ử hệ số nghiêm trọng
50 # A có trị riêng 1 và -17 (ổph ớn)
51 # -----
52 A_bad = np.array([[-49.0, 24.0],
53                  [-64.0, 31.0]])
54 ref_bad = expm(A_bad)
55
56 print("\nVí dụ 2 -- Ma trận gây ử hệ số A = [[-49,24],[-64,31]]")
57 print(f"  Két quả chuẩn xác: \n{ref_bad}\n")
58 print("  Bậc q ||Taylor - chính xác||_F  Max|lỗi trung gian|")

```

```

)
59 for q in [10, 20, 40, 59]:
60     approx = expm_taylor(A_bad, q)
61     err = np.linalg.norm(approx - ref_bad, 'fro')
62     # Tính giá trị ln ảnh hưởng của tổng trung gian (theo dõi lũy thừa)
63     n = A_bad.shape[0]
64     Ak = np.eye(n); fact = 1.0; max_elem = 0.0
65     for k in range(1, q + 1):
66         Ak = Ak @ A_bad; fact *= k
67         max_elem = max(max_elem, np.max(np.abs(Ak / fact)))
68     print(f"    q={q:3d}    {err:.2e}                                {max_elem:.2e}")
69
70 # -----
71 # Ví dụ 3: ước lượng Horner cho đa thức ma trận
72 # p(x) = 1 + x + x^2/2 + x^3/6 (Taylor bậc 3 của e^x)
73 # -----
74 A_small = np.array([[0.1, 0.2],
75                     [0.0, 0.3]])
76 coeffs = [1.0, 1.0, 0.5, 1/6] # hệ số [b0, b1, b2, b3]
77 p_horner = horner(A_small, coeffs)
78 p_direct = (np.eye(2) + A_small
79             + A_small @ A_small / 2
80             + A_small @ A_small @ A_small / 6)
81 print("\nVí dụ 3 -- Kiểm tra Horner vs tính trực tiếp:")
82 print(f"    ||Horner - tính trực tiếp||_F = {np.linalg.norm(p_horner
83     - p_direct):.2e}")

```

Listing 1.2: Phương pháp xấp xỉ: Taylor cắt cụt, Horner, và hiện tượng catastrophic cancellation

1.3 Hàm mũ ma trận (The Matrix Exponential)

Hàm mũ ma trận

$$e^{At} = \sum_{k=0}^{\infty} \frac{(At)^k}{k!}$$

là một trong những hàm ma trận được tính toán nhiều nhất trong thực tế, đặc biệt trong lý thuyết điều khiển và giải phương trình vi phân.

1.3.1 Phương pháp xấp xỉ Padé kết hợp Scaling and Squaring

Hàm Padé

Xấp xỉ Padé (p, q) cho hàm mũ được định nghĩa bởi:

$$R_{pq}(z) = D_{pq}(z)^{-1} N_{pq}(z),$$

trong đó

$$N_{pq}(z) = \sum_{k=0}^p \frac{(p+q-k)! p!}{(p+q)! k! (p-k)!} z^k, \quad D_{pq}(z) = \sum_{k=0}^q \frac{(p+q-k)! q!}{(p+q)! k! (q-k)!} (-z)^k.$$

Trường hợp $q = 0$: $R_{p0}(z) = 1 + z + \cdots + z^p/p!$ (đa thức Taylor bậc p).

Hạn chế và cách khắc phục

Xấp xỉ Padé chỉ chính xác gần gốc. Khai triển sai số:

$$e^A = R_{pq}(A) + \frac{(-1)^q}{(p+q)!} A^{p+q+1} D_{pq}(A)^{-1} \int_0^1 u^p (1-u)^q e^{A(1-u)} du.$$

Khắc phục bằng kỹ thuật *thu nhỏ và bình phương* (scaling and squaring):

$$e^A = (e^{A/m})^m, \quad m = 2^j \text{ (chọn } j \text{ phù hợp)}.$$

Tính $F_{pq} = R_{pq}(A/2^j) \approx e^{A/2^j}$, rồi bình phương j lần: $e^A \approx F_{pq}^{2^j}$.

Chặn sai số

Nếu $\|A\|_\infty / 2^j \leq 1/2$, tồn tại E sao cho $F_{pq} = e^{A+E}$, $AE = EA$, và

$$\|E\|_\infty \leq \varepsilon(p, q) \|A\|_\infty, \quad \varepsilon(p, q) = 2^{3-(p+q)} \frac{p! q!}{(p+q)! (p+q+1)!}.$$

Thuật toán 1.3.1 (Scaling and Squaring)

Nhập: $\delta > 0$, $A \in \mathbb{R}^{n \times n}$. Xuất: $F \approx e^A$ với sai số $\leq \delta \|A\|_\infty$.

1. $j = \max\{0, 1 + \lfloor \log_2 \|A\|_\infty \rfloor\}$; $A \leftarrow A/2^j$.
2. Chọn q nhỏ nhất sao cho $\varepsilon(q, j) \leq \delta$.
3. Tính $N = N_{qq}(A)$ và $D = D_{qq}(A)$ bằng Horner.
4. Giải $DF = N$ bằng phép khử Gauss.
5. Bình phương j lần: $F \leftarrow F^2$ (j lần).

Độ phức tạp: $\approx 2(q + j + 1/3)n^3$ flops.

1.3.2 Tối ưu hóa tính đa thức Padé

Dùng kỹ thuật Horner đặc biệt của §1.2.4, ví dụ với $q = 8$:

$$N_{88}(A) = U + AV, \quad D_{88}(A) = U - AV,$$

với $U = c_0 I + c_2 A^2 + (c_4 I + c_6 A^2 + c_8 A^4) A^4$, $V = c_1 I + c_3 A^2 + (c_5 I + c_7 A^2) A^4$. Chỉ cần 5 phép nhân thay vì 7.

1.3.3 Lý thuyết nhiễu loạn của hàm mũ ma trận

Xuất phát điểm là bài toán giá trị ban đầu:

$$\dot{X}(t) = AX(t), \quad X(0) = I \implies X(t) = e^{At}.$$

Đồng nhất thức quan trọng:

$$e^{(A+E)t} - e^{At} = \int_0^t e^{A(t-s)} E e^{(A+E)s} ds.$$

Chặn tăng trưởng qua phân tích Schur

Gọi $Q^H A Q = \text{diag}(\lambda_i) + N$ là phân tích Schur. Thì:

$$\|e^{At}\|_2 \leq e^{\alpha(A)t} M_S(t), \quad M_S(t) = \sum_{k=0}^{n-1} \frac{(\|N\|_2 t)^k}{k!},$$

trong đó $\alpha(A) = \max\{\text{Re}(\lambda) : \lambda \in \lambda(A)\}$ là *hoành độ phổ* (spectral abscissa). Nếu A là chính tắc ($N = 0$), thì $M_S(t) \equiv 1$ và $\|e^{At}\|_2 = e^{\alpha(A)t}$.

Số điều kiện của hàm mũ ma trận

Definition 1.3.1. Số điều kiện $\nu(A, t)$

$$\nu(A, t) = \max_{\|E\| \leq 1} \frac{\left\| \int_0^t e^{A(t-s)} E e^{As} ds \right\|_2}{\|A\|_2 \|e^{At}\|_2}.$$

Tính chất: $\nu(A, t) \geq t \|A\|_2$, với đẳng thức khi và chỉ khi A chính tắc. Nếu $\nu(A, t)$ lớn, thay đổi nhỏ trong A có thể gây thay đổi lớn trong e^{At} .

1.3.4 pseudospectra và hành vi quá độ

Nếu A không chính tắc, hàm mũ có thể *tăng trước khi suy giảm* dù mọi trị riêng đều có phần thực âm. Ví dụ minh họa:

$$A = \begin{pmatrix} -1 & 1000 \\ 0 & -1 \end{pmatrix} \implies e^{At} = e^{-t} \begin{pmatrix} 1 & 1000t \\ 0 & 1 \end{pmatrix}.$$

$\|e^{At}\|_2$ đạt đỉnh ≈ 500 tại $t \approx 1$ dù $\alpha(A) = -1 < 0$.
pseudospectra cho chặn dưới hữu dụng: với mọi $\varepsilon > 0$,

$$\sup_{t>0} \|e^{At}\|_2 \geq \frac{\alpha_\varepsilon(A)}{\varepsilon},$$

trong đó $\alpha_\varepsilon(A) = \sup_{z \in \Lambda_\varepsilon(A)} \text{Re}(z)$ là *hoành độ pseudospectra* ε .

1.3.5 Phân tích ổn định của thuật toán Scaling and Squaring

Vấn đề trong bước bình phương: nếu $\|e^{At}\|$ tăng mạnh ban đầu, sai số làm tròn tích lũy qua các lần bình phương:

$$\gamma = u \cdot \|G^2\|_2 \cdot \|G^4\|_2 \cdots \|G^{2^{j-1}}\|_2.$$

Nếu có tăng trưởng ban đầu lớn, $\gamma \gg u \|e^A\|_2$, tức là không thể đảm bảo sai số tương đối nhỏ. Với A chính tắc, $\gamma \approx u \|e^A\|_2$: thuật toán luôn cho sai số tương đối nhỏ.

1.3.6 Mã nguồn Python minh họa

Mã dưới đây cài đặt thuật toán Scaling and Squaring với xấp xỉ Padé bậc (6, 6), sau đó minh họa hiện tượng tăng trưởng quá độ với ma trận không chính tắc.

```

1 import numpy as np
2 from scipy.linalg import expm
3 from math import factorial
4
5 # -----
6 # ấXp ix Padé (p,p) cho hàm ũm
7 # -----
8 def pade_coeff(p):
9     """ảTr ềv ệh ốs ùca N_pp(z) và D_pp(z)."""
10    c = np.zeros(p + 1)
11    for k in range(p + 1):
12        c[k] = (factorial(p + p - k) * factorial(p)
13                / (factorial(p + p) * factorial(k) * factorial(p -
14                    k)))
15    # N_pp(z) = sum c[k] z^k
16    # D_pp(z) = sum c[k] (-z)^k => ệ h ốs: c[k]*(-1)^k
17    N_coeffs = c.copy()
18    D_coeffs = c * np.array([(-1)**k for k in range(p + 1)])
19    return N_coeffs, D_coeffs
20
21 def poly_matrix(A, coeffs):
22     """Tính đả úthc ma ậtrn ằbng Horner: ồtng theo k ùca coeffs[k]
23     A^k."""
24     n = A.shape[0]
25     q = len(coeffs) - 1
26     F = coeffs[q] * np.eye(n)
27     for k in range(q - 1, -1, -1):
28         F = F @ A + coeffs[k] * np.eye(n)
29     return F
30
31 def expm_scaling_squaring(A, p=6):
32     """
33     Scaling and Squaring óvi ấxp ix Padé ậbc (p,p).
34     - Thu ồnh: ợchn j sao cho ||A/2^j||_inf <= 0.5
35     - Tính Padé: R = D^{-1} N
36     - Bình ươphng j ầln
37     """
38     A = np.array(A, dtype=float)
39     norm_inf = np.max(np.sum(np.abs(A), axis=1))
40     j = max(0, int(np.ceil(np.log2(norm_inf / 0.5)))) if norm_inf >
41         0.5 else 0
42     As = A / (2 ** j) # thu ồnh
43
44     N_c, D_c = pade_coeff(p)
45     N = poly_matrix(As, N_c)
46     D = poly_matrix(As, D_c)
47     F = np.linalg.solve(D, N) # F = D^{-1} N

```

```

45     for _ in range(j):
46         F = F @ F
47     return F
48
49
50 # -----
51 # So sánh với SciPy expm
52 # -----
53 A_test = np.array([[2.0, 1.0, 0.0],
54                   [0.0, -1.0, 3.0],
55                   [0.0, 0.0, 0.5]])
56 F_mine = expm_scaling_squaring(A_test, p=6)
57 F_scipy = expm(A_test)
58 err = np.linalg.norm(F_mine - F_scipy, 'fro')
59 print("So sánh Scaling-Squaring (Padé p=6) vs SciPy expm:")
60 print(f" ||F_mine - F_scipy||_F = {err:.2e}")
61
62 # -----
63 # Hình tượng tăng trưởng quá độ
64 # A = [[-1, 1000], [0, -1]], trị riêng đều = -1 (âm)
65 # nhìn ||e^At||_2 tăng mạnh trước khi giảm
66 # -----
67 A_nt = np.array([[-1.0, 1000.0],
68                 [ 0.0, -1.0]])
69
70 t_vals = np.linspace(0, 3, 300)
71 norms = []
72 for t in t_vals:
73     eAt = expm(A_nt * t)
74     norms.append(np.linalg.norm(eAt, 2))
75
76 idx_max = np.argmax(norms)
77 print("\nHình tượng tăng trưởng quá độ vì A = [[-1,1000],[0,-1]]:"
78     )
79 print(f" Giá trị lớn nhất của ||e^At||_2 = {norms[idx_max]:.1f}")
80 print(f" Đạt tại t = {t_vals[idx_max]:.3f}")
81 print(f" (dù rằng alpha(A) = -1 < 0 nên e^At phải giảm trong dài
82     ạn)")
83
84 # Kiểm tra công thức giải tích
85 # e^{At} = e^{-t} [[1, 1000t], [0, 1]]
86 t_check = t_vals[idx_max]
87 eAt_analytic = np.exp(-t_check) * np.array([[1, 1000*t_check],
88                                             [0, 1]])
89 print(f" Kiểm tra: ||e^At||_2 (giải tích) = "
90     f"{np.linalg.norm(eAt_analytic, 2):.1f}")

```

Listing 1.3: Hàm mũ ma trận: Scaling and Squaring + Padé, và hiện tượng transient growth

1.4 Hàm dấu, căn bậc hai và logarit ma trận

1.4.1 Hàm dấu ma trận (Matrix Sign Function)

Định nghĩa

Với $z \in \mathbb{C}$ không nằm trên trục ảo:

$$\text{sign}(z) = \begin{cases} -1 & \text{nếu } \text{Re}(z) < 0, \\ +1 & \text{nếu } \text{Re}(z) > 0. \end{cases}$$

Giả sử $A \in \mathbb{C}^{n \times n}$ không có trị riêng thuần ảo, JCF được sắp xếp:

$$J = \begin{pmatrix} J_1 & 0 \\ 0 & J_2 \end{pmatrix},$$

$\lambda(J_1)$ nằm trong nửa phẳng trái, $\lambda(J_2)$ nằm trong nửa phẳng phải. Thì:

$$\text{sign}(A) = X \begin{pmatrix} -I_{m_1} & 0 \\ 0 & I_{m_2} \end{pmatrix} X^{-1}.$$

Ứng dụng tính không gian bất biến

Với phân hoạch $X = [X_1 \mid X_2]$, $X^{-H} = [Y_1 \mid Y_2]$:

$$\frac{1}{2}(I + \text{sign}(A)) = X_2 Y_2^H \quad (\text{rank } m_2).$$

Phân tích QR-với-đổi cột của ma trận này cho xấp xỉ không gian bất biến gần với các trị riêng nửa phẳng phải.

Thuật toán lặp Newton

Hàm dấu là nghiệm của $g(z) = z^2 - 1 = 0$. Tương tự Newton vô hướng:

$$S_0 = A; \quad S_{k+1} = \frac{1}{2}(S_k + S_k^{-1}), \quad k = 0, 1, \dots$$

Theorem 1.4.1. Hội tụ Newton sign

Nếu A không có trị riêng thuần ảo, dãy $\{S_k\}$ xác định với mọi k , $\text{sign}(S_k) = \text{sign}(A)$ cho mọi k , và $S_k \rightarrow \text{sign}(A)$ với tốc độ bậc hai:

$$\|S_{k+1} - S\| \leq \frac{1}{2} \|S_k^{-1}\| \cdot \|S_k - S\|^2.$$

Bằng chứng dựa trên $G_k = (S_k - S)(S_k + S)^{-1}$ thỏa $G_{k+1} = G_k^2 \rightarrow 0$.

Biến thể và gia tốc**Lặp Newton-Schulz:** Dùng xấp xỉ $S_k^{-1} \approx S_k(2I - S_k^2)$:

$$S_{k+1} = \frac{1}{2}S_k(3I - S_k^2).$$

Không cần tính nghịch đảo ma trận, nhưng chỉ hội tụ cục bộ.

Lặp có hệ số tỷ lệ:

$$S_{k+1} = \frac{1}{2}(\mu_k S_k + (\mu_k S_k)^{-1}),$$

với các lựa chọn $\mu_k = |\det(S_k)|^{1/n}$ hoặc $\mu_k = \sqrt{\rho(S_k^{-1})/\rho(S_k)}$.**1.4.2 Căn bậc hai ma trận (Matrix Square Root)****Sự không đơn trị**

Ma trận có thể có nhiều căn bậc hai. Ví dụ:

$$A = \begin{pmatrix} 4 & 10 \\ 0 & 9 \end{pmatrix} \text{ có ít nhất 4 căn bậc hai: } \begin{pmatrix} \pm 2 & * \\ 0 & \pm 3 \end{pmatrix}.$$

Definition 1.4.1. Căn bậc hai chính (Principal Square Root)

$F = A^{1/2}$ là căn bậc hai chính của A nếu $F^2 = A$ và mọi trị riêng của F đều có phần thực dương.

Lặp Newton cho căn bậc haiTương tự Newton vô hướng $x_{k+1} = (x_k + a/x_k)/2$:

$$X_0 = A; \quad X_{k+1} = \frac{1}{2}(X_k + X_k^{-1}A), \quad k = 0, 1, \dots$$

Thay $X_k = A^{1/2}S_k$ vào trên cho ra lặp Newton sign cho $A^{1/2}$: hội tụ toàn cục và bậc hai.**Mối liên hệ với hàm dấu ma trận**

Áp dụng lặp Newton sign cho ma trận ghép:

$$\tilde{A} = \begin{pmatrix} 0 & A \\ I & 0 \end{pmatrix}.$$

Qua quy nạp, các vòng lặp $\tilde{S}_k$ có dạng $\begin{pmatrix} 0 & X_k \\ Y_k & 0 \end{pmatrix}$, và:

$$\text{sign} \begin{pmatrix} 0 & A \\ I & 0 \end{pmatrix} = \begin{pmatrix} 0 & A^{1/2} \\ A^{-1/2} & 0 \end{pmatrix}.$$

Lặp Denman-Beavers

Từ quan hệ trên, lặp Denman-Beavers có thuộc tính số tốt hơn Newton:

$$X_{k+1} = \frac{X_k + Y_k^{-1}}{2}, \quad Y_{k+1} = \frac{Y_k + X_k^{-1}}{2},$$

với $X_0 = A, Y_0 = I$. Dãy này hội tụ $X_k \rightarrow A^{1/2}, Y_k \rightarrow A^{-1/2}$.

1.4.3 Phân tích cực (Polar Decomposition)

Định lý cơ bản

Theorem 1.4.2. Phân tích cực

Nếu $A \in \mathbb{R}^{m \times n}, m \geq n$, thì tồn tại $U \in \mathbb{R}^{m \times n}$ (cột trực chuẩn) và $P \in \mathbb{R}^{n \times n}$ (hữu xác định dương đối xứng) sao cho $A = UP$.

Chứng minh dựa trên SVD mỏng: $A = U_A \Sigma_A V_A^T$ cho $U = U_A V_A^T, P = V_A \Sigma_A V_A^T$. Ta có $P = (A^T A)^{1/2}$ và nếu $\text{rank}(A) = n$ thì $U = A(A^T A)^{-1/2}$.

Thuật toán lặp Newton cho nhân tử trực giao

$$X_0 = A; \quad X_{k+1} = \frac{X_k + X_k^{-T}}{2}, \quad k = 0, 1, \dots$$

Nếu $X_k = U_k P_k$ (phân tích cực của X_k), thì

$$X_{k+1} = U_k \cdot \frac{P_k + P_k^{-1}}{2}.$$

Do $P_{k+1} = (P_k + P_k^{-1})/2$ đây chính là lặp Newton sign cho $P_0 = P$. Vì $P_k \rightarrow \text{sign}(P) = I$ bậc hai, nên $X_k \rightarrow U$ bậc hai.

Kết nối với hàm dấu

$$\text{sign} \begin{pmatrix} 0 & A \\ A^T & 0 \end{pmatrix} = \begin{pmatrix} 0 & U \\ U^T & 0 \end{pmatrix},$$

trong đó $U = U_A V_A^T$ là nhân tử trực giao của A .

Lý thuyết nhiễu loạn

Theorem 1.4.3. Nhiễu loạn phân tích cực – Li và Sun (2003)

Nếu $A, \tilde{A} \in \mathbb{R}^{n \times n}$ không suy biến với nhân tử trực giao U và $\tilde{U}$ tương ứng, thì

$$\|U - \tilde{U}\|_F \leq \frac{4 \|A - \tilde{A}\|_F}{\sigma_{n-1}(A) + \sigma_n(A) + \sigma_{n-1}(\tilde{A}) + \sigma_n(\tilde{A})}.$$

1.4.4 Logarit ma trận (Matrix Logarithm)**Định nghĩa và tính không đơn trị**

Nếu $e^X = A$, thì X là một *logarit* của A . Do tính tuần hoàn của hàm mũ phức, $X + 2k\pi i$ cũng là logarit. Để khử mập mờ:

Definition 1.4.2. Logarit chính (Principal Logarithm)

Nếu $A \in \mathbb{R}^{n \times n}$ có mọi trị riêng thực đều dương, thì tồn tại duy nhất ma trận thực X thỏa $e^X = A$ với $\lambda(X) \subset \{z \in \mathbb{C} : -\pi < \text{Im}(z) < \pi\}$.

Các khai triển chuỗi cho hàm log**Chuỗi Maclaurin:**

$$\log(A) \approx M_q(A) = \sum_{k=1}^q \frac{(-1)^{k+1} (A - I)^k}{k}, \quad \rho(A - I) < 1.$$

Chuỗi Gregory:

$$\log(A) \approx G_q(A) = -2 \sum_{k=0}^q \frac{1}{2k+1} [(I - A)(I + A)^{-1}]^{2k+1}, \quad \text{Re}(\lambda_i) > 0.$$

Xấp xỉ Padé chéo: Ví dụ xấp xỉ (3, 3):

$$\begin{aligned} \log(A) &\approx r_{33}(A) = D(A)^{-1}N(A), \\ D(A) &= 60I + 90(A - I) + 36(A - I)^2 + 3(A - I)^3, \\ N(A) &= 60(A - I) + 60(A - I)^2 + 11(A - I)^3. \end{aligned}$$

Thuật toán Inverse Scaling and Squaring**Thuật toán 1.4.1 (Inverse Scaling and Squaring cho $\log(A)$)**

1. Lấy căn bậc hai lặp cho đến $\|A_k - I\| \leq \text{tol}$:

$$k = 0, \quad A_0 = A; \quad \text{Khi } \|A - I\| > \text{tol} : k \leftarrow k + 1, \quad A_k = A_{k-1}^{1/2}.$$

(Dùng lặp Denman-Beavers để tính căn bậc hai.)

2. Tính $F \approx \log(A_k)$ bằng xấp xỉ Padé phù hợp.

3. Hoàn nguyên: $\log(A) = 2^k F$.

Đây là quá trình ngược của Scaling and Squaring cho hàm mũ.

1.4.5 Mã nguồn Python minh họa

Mã nguồn sau minh họa bốn thuật toán: (1) lặp Newton cho hàm dấu ma trận, (2) lặp Denman-Beavers cho căn bậc hai, (3) lặp Newton cho nhân tử trực giao (phân tích cực), và (4) thuật toán Inverse Scaling and Squaring cho logarit.

```

1 import numpy as np
2 from scipy.linalg import sqrtm, logm, svd
3
4 # -----
5 # 1. Lặp Newton cho hàm dấu ma trận
6 #    $S_{k+1} = 0.5*(S_k + S_k^{-1})$ 
7 # -----
8 def matrix_sign_newton(A, max_iter=50, tol=1e-14):
9     S = A.astype(complex).copy()
10    for k in range(max_iter):
11        S_inv = np.linalg.inv(S)
12        S_new = 0.5 * (S + S_inv)
13        residual = np.linalg.norm(S_new - S, 'fro')
14        if residual < tol:
15            print(f" [sign] ộHi ụt sau {k+1} vòng ặp, ầphn ud = {
16                residual:.2e}")
17            return S_new
18        S = S_new
19    print(f" [sign] ưCha ộhi ụt sau {max_iter} vòng ặp")
20    return S
21
22 # -----
23 # 2. Lặp Denman-Beavers cho ăcn ậbc hai ma ậtrn
24 #    $X_{k+1} = 0.5*(X_k + Y_k^{-1})$ 
25 #    $Y_{k+1} = 0.5*(Y_k + X_k^{-1})$ 
26 # -----
27 def matrix_sqrt_denman_beavers(A, max_iter=50, tol=1e-14):
28     X = A.astype(float).copy()
29     Y = np.eye(A.shape[0])
30     for k in range(max_iter):
31         X_inv = np.linalg.inv(X)
32         Y_inv = np.linalg.inv(Y)
33         X_new = 0.5 * (X + Y_inv)
34         Y_new = 0.5 * (Y + X_inv)
35         err = np.linalg.norm(X_new @ X_new - A, 'fro')
36         if err < tol:
37             print(f" [sqrt] ộHi ụt sau {k+1} vòng ặp, ||X^2-A||_F
38                 = {err:.2e}")
39             return X_new, Y_new
40         X, Y = X_new, Y_new
41     return X, Y
42
43 # -----

```

```

42 # 3. Phân tích ực: Newton ălp cho nhân ừt ựtrc giao
43 #    $X_{k+1} = 0.5*(X_k + X_k^{-T})$ 
44 # -----
45 def polar_newton(A, max_iter=50, tol=1e-14):
46     X = A.astype(float).copy()
47     for k in range(max_iter):
48         X_invT = np.linalg.inv(X).T
49         X_new = 0.5 * (X + X_invT)
50         err = np.linalg.norm(X_new.T @ X_new - np.eye(A.shape[0]),
51                               'fro')
52         if err < tol:
53             print(f" [polar] ộHi ựt sau {k+1} vòng ălp, ||U^T U -
54                   I||_F = {err:.2e}")
55             return X_new
56         X = X_new
57     return X
58 # -----
59 # 4. Logarit ma ậtrn: Inverse Scaling and Squaring
60 #   Dùng ăcn ậbc hai ălp + Maclaurin cho log
61 # -----
62 def matrix_log_iss(A, tol_norm=0.5, q=20):
63     """Inverse Scaling and Squaring cho logm."""
64     Ak = A.astype(float).copy()
65     k = 0
66     while np.linalg.norm(Ak - np.eye(Ak.shape[0]), 'fro') >
67           tol_norm:
68         Ak, _ = matrix_sqrt_denman_beavers(Ak, tol=1e-13)
69         k += 1
70     # Tính log(Ak) ằbng ỗchui Maclaurin (Ak ằgn I)
71     B = Ak - np.eye(Ak.shape[0])
72     logA = np.zeros_like(B)
73     Bk = np.eye(Ak.shape[0])
74     for j in range(1, q + 1):
75         Bk = Bk @ B
76         logA += ((-1)**(j+1) / j) * Bk
77     return (2**k) * logA # hoàn nguyên
78 # -----
79 # ỨNG DỤNG VÀ ỀKIM TRA
80 # -----
81 A = np.array([[4.0, 3.0],
82               [1.0, 2.0]])
83 print("Ma ậtrn A:")
84 print(A)
85
86 # Hàm ấdu
87 S = matrix_sign_newton(A)
88 print("\nHàm ấdu sign(A):")
89 print(S.real)

```

```

90
91 # ăCn ậbc hai
92 X_sqrt, _ = matrix_sqrt_denman_beavers(A)
93 ref_sqrt = sqrtm(A)
94 print("\năCn ậbc hai A^{1/2} (Denman-Beavers):")
95 print(X_sqrt)
96 print(f" Sai ốs so óvi sqrtm(scipy): {np.linalg.norm(X_sqrt -
    ref_sqrt, 'fro'):.2e}")
97
98 # Phân tích ựcc
99 U_polar = polar_newton(A)
100 U_svd = (lambda U,_,Vt: U @ Vt)(*svd(A, full_matrices=False))
101 print("\nNhân ửt ựtrc giao U (phân tích ựcc, Newton):")
102 print(U_polar)
103 print(f" Sai ốs so óvi SVD: {np.linalg.norm(U_polar - U_svd, 'fro
    '):.2e}")
104
105 # Logarit
106 L = matrix_log_iss(A, tol_norm=0.5, q=30)
107 ref_log = logm(A)
108 print("\nLogarit chính log(A) (Inverse Scaling and Squaring):")
109 print(L)
110 print(f" Sai ốs so óvi logm(scipy): {np.linalg.norm(L - ref_log, '
    fro'):.2e}")
111 print(f" ể Kim tra: ||e^L - A||_F = "
112       f"{np.linalg.norm(np.linalg.matrix_power(np.eye(2), 0) + L -
    A, 'fro'):.2e}")

```

Listing 1.4: Hàm dấu, căn bậc hai, phân tích cực và logarit ma trận – lặp Newton và Denman-Beavers

Chương 2

Triển khai thực nghiệm

Chương này trình bày các thực nghiệm tính toán cụ thể, đánh giá định lượng hiệu năng và độ chính xác của các phương pháp đã trình bày ở Chương 1. Tất cả thực nghiệm được thực hiện bằng Python 3.10 với các thư viện NumPy 1.25 và SciPy 1.11 trên môi trường chuẩn IEEE 754 double precision ($u \approx 2.22 \times 10^{-16}$).

2.1 Môi trường và thiết lập thực nghiệm

2.1.1 Yêu cầu thư viện

Toàn bộ mã thực nghiệm sử dụng các thư viện Python phổ biến, dễ dàng cài đặt:

```
1 # Cài đặt uth ệvin (ếnu urchase có)
2 # pip install numpy scipy matplotlib
3
4 import numpy as np
5 from scipy.linalg import (expm, logm, sqrtm,
6                           schur, svd, norm)
7 import time
8
9 # ếThit ậlp random seed để tái ệhin ếkt ậqu
10 np.random.seed(42)
11
12 # Epsilon máy
13 u = np.finfo(float).eps
14 print(f"Epsilon máy u = {u:.3e}")
15 print(f"Phiên ậbn NumPy: {np.__version__}")
16
17 # Hàm ậto ma ậtrn ậngu nhiên óvi ós điều ệkin cho uótrc
18 def random_matrix_cond(n, kappa):
19     """ậto ma ậtrn ậngu nhiên n x n óvi condition number ậxp ix
20     kappa."""
21     U, _ = np.linalg.qr(np.random.randn(n, n))
22     V, _ = np.linalg.qr(np.random.randn(n, n))
23     s = np.logspace(0, -np.log10(kappa), n)
24     return U @ np.diag(s) @ V.T
25
26 # Hàm đo sai ós uótrng đối Frobenius
27 def rel_err(F_approx, F_exact):
28     denom = norm(F_exact, 'fro')
29     return norm(F_approx - F_exact, 'fro') / denom if denom > 0
30     else float('inf')
```

Listing 2.1: Thiết lập môi trường thực nghiệm

2.1.2 Các ma trận thử nghiệm chuẩn

Để đánh giá toàn diện, chúng ta dùng bốn lớp ma trận thử nghiệm đại diện cho các trường hợp khó khác nhau trong thực tế.

```

1 def test_matrices():
2     """Trả về dict các ma trận thử nghiệm chuẩn."""
3     mats = {}
4
5     # 1. Ma trận chéo hóa được, điều kiện tốt
6     mats['diagonal'] = np.diag([1.0, 2.0, 3.0, 4.0])
7
8     # 2. Ma trận đối xứng xác định dương (SPD)
9     B = np.random.randn(4, 4)
10    mats['SPD'] = B.T @ B + 4 * np.eye(4)
11
12    # 3. Ma trận có trị riêng phức (xoay)
13    theta = np.pi / 4
14    mats['rotation'] = np.array([
15        [ np.cos(theta), -np.sin(theta), 0, 0],
16        [ np.sin(theta),  np.cos(theta), 0, 0],
17        [ 0, 0,  np.cos(theta), -np.sin(theta)],
18        [ 0, 0,  np.sin(theta),  np.cos(theta)]
19    ])
20
21    # 4. Ma trận không chính tắc (non-normal) - khá khó
22    mats['non_normal'] = np.array([
23        [-1.0, 500.0, 0.0, 0.0],
24        [ 0.0, -1.0, 500.0, 0.0],
25        [ 0.0,  0.0, -1.0, 500.0],
26        [ 0.0,  0.0,  0.0, -1.0]
27    ])
28
29    # 5. Ma trận trị riêng kép (Jordan block 4x4)
30    lam = 2.0
31    mats['jordan'] = lam * np.eye(4) + np.diag([1, 1, 1], k=1)
32
33    return mats
34
35 MATS = test_matrices()
36 for name, M in MATS.items():
37     kappa = np.linalg.cond(M)
38     eigvals = np.linalg.eigvals(M)
39     print(f"  {name:15s}: kích thước={M.shape}, "
40           f"kappa={kappa:.2e}, "
41           f"trị riêng ~ {np.round(eigvals, 2)}")

```

Listing 2.2: Định nghĩa các ma trận thử nghiệm

2.2 Thực nghiệm 1: Phương pháp trị riêng và Schur-Parlett

Thực nghiệm này so sánh ba phương pháp tính e^A : phân tích trị riêng (eigenvector), Schur-Parlett, và SciPy chuẩn. Chúng ta đo sai số tương đối Frobenius và thời gian chạy.

```

1 import numpy as np
2 from scipy.linalg import expm, schur
3 import time
4
5 def expm_eigenvector(A):
6     """Tính  $e^A$  qua phân tích trị riêng."""
7     lam, X = np.linalg.eig(A)
8     return X @ np.diag(np.exp(lam)) @ np.linalg.inv(X)
9
10 def expm_schur_parlett(A):
11     """
12     Tính  $e^A$  qua Schur-Parlett (n ăgin hóa cho trị riêng phân
13     ệbit).
14      $A = Q T Q^H \Rightarrow e^A = Q e^T Q^H$ 
15     """
16     T, Q = schur(A, output='complex')
17     n = T.shape[0]
18     F = np.zeros_like(T, dtype=complex)
19     lam = np.diag(T)
20
21     # ở Khi ăto đường chéo
22     for i in range(n):
23         F[i, i] = np.exp(lam[i])
24
25     # Tính các siêu đường chéo theo úth ựt  $p = 1, 2, \dots, n-1$ 
26     for p in range(1, n):
27         for i in range(n - p):
28             j = i + p
29             dl = lam[j] - lam[i]
30             if abs(dl) > 1e-10:
31                 # Sai phân chia ăbc 1
32                 Fij = T[i, j] * (F[j, j] - F[i, i]) / dl
33             else:
34                 # ó Gii ăhn:  $f'(\lambda)$  khi trị riêng trùng nhau
35                 Fij = T[i, j] * np.exp(lam[i])
36             # óng gó ừt các ăphn ừt trung gian
37             for k in range(i + 1, j):
38                 dlk = lam[j] - lam[k]
39                 dki = lam[k] - lam[i]
40                 if abs(dlk) > 1e-10 and abs(dki) > 1e-10:
41                     Fij += (T[i, k] * F[k, j] - F[i, k] * T[k, j]) / dl
42             F[i, j] = Fij
43
44     return (Q @ F @ Q.conj().T).real
45 # -----

```

```

46 # ậChy ừth ệngheim trên các ma ậtrn ầchun
47 # -----
48 print(f"{'Ma ậtrn':15s} | {'Sai ốs Eig':10s} | {'Sai ốs Schur':10s}
    "
49       f"| {'t_eig (ms)':11s} | {'t_schur (ms)':12s}")
50 print("-" * 68)
51
52 for name, A in MATS.items():
53     ref = expm(A)
54
55     t0 = time.perf_counter()
56     F_eig = expm_eigenvector(A).real
57     t_eig = (time.perf_counter() - t0) * 1000
58
59     t0 = time.perf_counter()
60     F_sch = expm_schur_parlett(A)
61     t_sch = (time.perf_counter() - t0) * 1000
62
63     e_eig = rel_err(F_eig, ref)
64     e_sch = rel_err(F_sch, ref)
65
66     print(f"{name:15s} | {e_eig:10.2e} | {e_sch:10.2e} "
67           f"| {t_eig:11.3f} | {t_sch:12.3f}")

```

Listing 2.3: Thực nghiệm 1: so sánh phương pháp trị riêng vs Schur-Parlett

2.3 Thực nghiệm 2: Phương pháp xấp xỉ – Taylor và ảnh hưởng của hủy số

Thực nghiệm này khảo sát kỹ lưỡng hiện tượng catastrophic cancellation trong xấp xỉ Taylor, và minh họa tại sao bậc xấp xỉ cao hơn không luôn tốt hơn.

```

1 import numpy as np
2 from scipy.linalg import expm
3
4 def expm_taylor(A, q):
5     n = A.shape[0]
6     F = np.eye(n); Ak = np.eye(n); fact = 1.0
7     for k in range(1, q + 1):
8         Ak = Ak @ A; fact *= k; F += Ak / fact
9     return F
10
11 def expm_taylor_scaled(A, q, j=None):
12     """Taylor ớvi scaling: tính e^{A/2^j} ờri bình ươphng j ầln."""
13     norm_A = np.max(np.sum(np.abs(A), axis=1))
14     if j is None:
15         j = max(0, int(np.ceil(np.log2(norm_A))))
16     As = A / (2 ** j)
17     F = expm_taylor(As, q)
18     for _ in range(j): F = F @ F

```

```

19     return F
20
21 # Ma trận gây nhiễu ổn định
22 A_bad = np.array([[ -49.0, 24.0],
23                  [-64.0, 31.0]])
24 ref = expm(A_bad)
25
26 print("Hệ phương trình ổn định với A = [[-49,24],[-64,31]]")
27 print(f"Kết quả chính xác: \n{ref}\n")
28
29 print(f"{'Bậc q':6s} | {'Taylor (err)':20s} | {'Taylor +\n\nScaling (err)':22s}")
30 print(" " + "-"*55)
31 for q in [5, 10, 15, 20, 30, 50, 59]:
32     F_naive = expm_taylor(A_bad, q)
33     F_scaled = expm_taylor_scaled(A_bad, q)
34     e_naive = rel_err(F_naive, ref)
35     e_scaled = rel_err(F_scaled, ref)
36     print(f" {q:6d} | {e_naive:20.4e} | {e_scaled:22.4e}")
37
38 # Theo dõi giá trị ln ổn định của các số hạng trung gian
39 print("\nGiá trị ln ổn định của các số hạng Ak/k! (A_bad):")
40 n = A_bad.shape[0]
41 Ak = np.eye(n); fact = 1.0
42 for k in range(1, 21):
43     Ak = Ak @ A_bad; fact *= k
44     term_max = np.max(np.abs(Ak / fact))
45     if k <= 5 or k % 5 == 0:
46         print(f" k={k:2d}: max|A^k / k!| = {term_max:.3e}")

```

Listing 2.4: Thực nghiệm 2: hiện tượng hủy số trong Taylor và vai trò của scaling

2.4 Thực nghiệm 3: Scaling and Squaring – phân tích hiệu năng theo kích thước

Thực nghiệm này đo thời gian chạy và độ chính xác của thuật toán Scaling and Squaring (SciPy expm) so với phương pháp trị riêng thuần túy khi kích thước ma trận n tăng.

```

1 import numpy as np
2 from scipy.linalg import expm
3 import time
4
5 def expm_eigenvector_general(A):
6     lam, X = np.linalg.eig(A)
7     return (X @ np.diag(np.exp(lam)) @ np.linalg.inv(X)).real
8
9 sizes = [4, 8, 16, 32, 64, 128, 256]
10 n_repeat = 5 # số lần lặp để lấy trung bình
11
12 print(f"{'n':>6} | {'t_eig (ms)':>12} | {'t_expm (ms)':>12} ")

```

```

13         f"| {'ăTng ốtc':>8} | {'sai ốs eig':>10}")
14 print("-" * 60)
15
16 for n in sizes:
17     # ậTo ma ậtrn ẫng nhiên xác định ương (luôn có e^A xác định)
18     B = np.random.randn(n, n)
19     A = (B + B.T) / 2 - n * np.eye(n)    # đối úxng, âm định
20
21     ref = expm(A)
22
23     # Đo ờthi gian expm (SciPy -- Padé + Scaling&Squaring)
24     t_expm_list = []
25     for _ in range(n_repeat):
26         t0 = time.perf_counter()
27         F_expm = expm(A)
28         t_expm_list.append((time.perf_counter() - t0) * 1000)
29     t_expm = np.median(t_expm_list)
30
31     # Đo ờthi gian ươphng pháp ịtr riêng (ích ảkh thi khi n ồnh)
32     if n <= 64:
33         t_eig_list = []
34         for _ in range(n_repeat):
35             t0 = time.perf_counter()
36             F_eig = expm_eigenvector_general(A)
37             t_eig_list.append((time.perf_counter() - t0) * 1000)
38         t_eig = np.median(t_eig_list)
39         err_eig = rel_err(F_eig, ref)
40         speedup = t_eig / t_expm
41         print(f"{n:>6} | {t_eig:>12.3f} | {t_expm:>12.3f} "
42               f"| {speedup:>8.2f}x | {err_eig:>10.2e}")
43     else:
44         print(f"{n:>6} | {'N/A (quá ớln)':>12} | {t_expm:>12.3f} "
45               f"| {'---':>8} | {'---':>10}")

```

Listing 2.5: Thực nghiệm 3: hiệu năng Scaling and Squaring theo kích thước n

2.5 Thực nghiệm 4: Hàm dấu, căn bậc hai và logarit – hội tụ và độ chính xác

Thực nghiệm cuối cùng phân tích chi tiết tốc độ hội tụ của lặp Newton và Denman-Beavers, đồng thời so sánh kết quả với hàm chuẩn SciPy.

```

1 import numpy as np
2 from scipy.linalg import sqrtm, logm, expm
3
4 # Cài đặt ặli các hàm ặlp ớvi theo đối ộhi ựt
5 def matrix_sign_newton_track(A, max_iter=30, tol=1e-14):
6     S = A.astype(complex).copy()
7     residuals = []
8     for k in range(max_iter):

```

```

9      S_new = 0.5 * (S + np.linalg.inv(S))
10     res    = np.linalg.norm(S_new @ S_new - np.eye(S.shape[0]),
11                             'fro')
12     residuals.append(res)
13     if res < tol: S = S_new; break
14     S = S_new
15     return S, residuals
16
17 def matrix_sqrt_db_track(A, max_iter=30, tol=1e-14):
18     X = A.astype(float).copy()
19     Y = np.eye(A.shape[0])
20     residuals = []
21     for k in range(max_iter):
22         X_inv = np.linalg.inv(X)
23         Y_inv = np.linalg.inv(Y)
24         X_new = 0.5 * (X + Y_inv)
25         Y_new = 0.5 * (Y + X_inv)
26         res    = np.linalg.norm(X_new @ X_new - A, 'fro')
27         residuals.append(res)
28         if res < tol: X, Y = X_new, Y_new; break
29         X, Y = X_new, Y_new
30     return X, Y, residuals
31
32 # -----
33 # Ma trận uth ệngheim: SPD, để kìm úchng
34 # -----
35 B = np.array([[4.0, 2.0, 1.0],
36               [2.0, 5.0, 3.0],
37               [1.0, 3.0, 6.0]])
38
39 print("=" * 55)
40 print("MA ẬTRN B =")
41 print(B)
42 print(f"ịTr riêng: {np.linalg.eigvals(B).real}")
43
44 # Thí ệngheim: Hàm ấdu sign(B)
45 print("\n--- Hàm ấdu ma ậtrn (Newton) ---")
46 S_B, res_sign = matrix_sign_newton_track(B)
47 ref_sign = np.diag(np.sign(np.linalg.eigvals(B).real))
48 # ưLu ý: sign(B) ựph ộthuc vào ịtr riêng ựthc đề(u ương => sign = I)
49 print(f"    sign(B) (ịtr riêng đều ương, nên sign(B) = I):")
50 print(f"    {np.diag(S_B.real)}")
51 print("    ầ Phn ứ ||S^2 - I||_F theo vòng ặlp:")
52 for k, r in enumerate(res_sign):
53     print(f"        Vòng {k+1}: {r:.4e}")
54
55 # Thí ệngheim: ấCn ậbc hai Denman-Beavers
56 print("\n--- ấCn ậbc hai ma ậtrn (Denman-Beavers) ---")
57 X_sqrt, _, res_sqrt = matrix_sqrt_db_track(B)
58 ref_sqrt = sqrtm(B)

```

```

58 err_sqrt = np.linalg.norm(X_sqrt - ref_sqrt, 'fro') / np.linalg.
    norm(ref_sqrt, 'fro')
59 print(f" Sai số so với sqrtm(scipy): {err_sqrt:.2e}")
60 print(" à Phn ưd ||X^2 - B||_F theo vòng ălp:")
61 for k, r in enumerate(res_sqrt):
62     print(f" Vòng {k+1}: {r:.4e}")
63
64 # Thí ệnhim: Logarit -- ềkim úchng  $e^{\log(B)} = B$ 
65 print("\n--- Logarit ma ậtrn (SciPy logm) ---")
66 L_B = logm(B)
67 check = expm(L_B)
68 err_log = np.linalg.norm(check - B, 'fro') / np.linalg.norm(B, 'fro')
69 print(f" Sai số ||e^{{logm(B)}} - B||_F / ||B||_F = {err_log:.2e}")
70
71 # -----
72 # Thí ệnhim phân tích ực óvi SVD đối ếchiu
73 # -----
74 print("\n--- Phân tích ực A = U P ---")
75 A_rect = np.array([[1.0, 2.0, 0.0],
76                   [0.0, 3.0, 4.0],
77                   [5.0, 0.0, 1.0]])
78 Ua, Sa, Vat = np.linalg.svd(A_rect)
79 U_polar_ref = Ua @ Vat # nhâ ừt ựtrc giao ừt SVD
80 P_polar_ref = Vat.T @ np.diag(Sa) @ Vat # nhâ ừt xác định ương
81
82 # Newton ălp cho nhâ ừt ựtrc giao
83 X = A_rect.copy()
84 for k in range(30):
85     X_new = 0.5 * (X + np.linalg.inv(X).T)
86     if np.linalg.norm(X_new - X, 'fro') < 1e-14: break
87     X = X_new
88 U_newton = X
89
90 err_polar = np.linalg.norm(U_newton - U_polar_ref, 'fro')
91 print(f" ||U_Newton - U_SVD||_F = {err_polar:.2e}")
92 print(f" ề Kim tra ựtrc giao: ||U^T U - I||_F = "
93       f"{np.linalg.norm(U_newton.T @ U_newton - np.eye(3), 'fro'):.2e}")
94 print(f" ề Kim tra phân tích: ||A - UP||_F = "
95       f"{np.linalg.norm(A_rect - U_newton @ P_polar_ref, 'fro'):.2e}")

```

Listing 2.6: Thực nghiệm 4: hội tụ bậc hai của lặp Newton và Denman-Beavers

2.6 Tổng hợp và nhận xét kết quả

Dựa trên bốn thực nghiệm trên, ta có thể đưa ra một số nhận xét tổng hợp.

2.6.1 So sánh tổng quan các phương pháp

Phương pháp	Độ chính xác	Ổn định số	Chi phí	Phù hợp khi
Trị riêng (eigenvector)	Phụ thuộc $\kappa_2(X)$	Kém nếu X ill-cond	$O(n^3)$	A chính tắc
Schur-Parlett	Cao	Tốt	$O(n^3)$	n nhỏ-vừa
Taylor cắt cụt	Bị hủy số	Kém	$O(qn^3)$	Chỉ để minh họa
Taylor + Scaling	Tốt	Khá	$O(qn^3)$	$\ A\ $ lớn
Padé + Scal. & Sq.	Cao	Rất tốt	$O(n^3)$	Mục đích chung
Newton sign	Bậc hai	Tốt	$O(kn^3)$	Hàm dấu, căn
Denman-Beavers	Bậc hai	Tốt hơn Newton	$O(kn^3)$	Căn bậc hai

2.6.2 Khuyến nghị thực tế

Khuyến nghị thực hành:

1. **Hàm mũ e^A :** Luôn dùng `scipy.linalg.expm` (Padé bậc 13 + Scaling and Squaring) trong thực tế. Chỉ tự cài đặt khi nghiên cứu hoặc yêu cầu đặc biệt về hiệu năng.
2. **Logarit $\log(A)$:** Dùng `scipy.linalg.logm` (Inverse Scaling and Squaring). Cần thận khi A có trị riêng âm hoặc phức (vấn đề chọn nhánh logarit).
3. **Căn bậc hai $A^{1/2}$:** `scipy.linalg.sqrtm` cho kết quả tốt. Lặp Denman-Beavers hữu ích khi cần kiểm soát trực tiếp quá trình hội tụ.
4. **Ma trận lớn ($n \gg 1000$):** Chuyển sang phương pháp Krylov (ví dụ: Expokit, Expmv) thay vì các phương pháp dense.
5. **Kiểm tra điều kiện:** Trước khi tính bất kỳ hàm ma trận nào, nên kiểm tra số điều kiện $\kappa_2(A)$ và hoành độ phổ $\alpha(A)$ để dự báo độ chính xác kết quả.

2.6.3 Tổng kết

Qua lý thuyết ở Chương 1 và thực nghiệm ở Chương 2, ta thấy rõ rằng không có một phương pháp duy nhất nào tối ưu cho mọi trường hợp. Sự lựa chọn phụ thuộc vào cấu trúc ma trận (chính tắc hay không, điều kiện tốt hay xấu, kích thước), hàm cần tính (e^A , $\log A$, $A^{1/2}$, $\text{sign}(A)$), và yêu cầu về độ chính xác và tốc độ. Tuy nhiên, các phương pháp dựa trên phân tích Schur và xấp xỉ Padé với Scaling and Squaring nổi bật là lựa chọn ổn định và hiệu quả nhất cho mục đích tổng quát, và đây cũng là nền tảng của các thư viện số hiện đại như MATLAB, SciPy và Julia.